

Project Proposal



A Multi-Tenant Private Social Network and Campus Platform for Universities

Prepared by — Team Mavericks:

Name	Student ID
Joydip Datta	0112330960
MD. Saem Ferdous	0112330950
Md. Monabbur Hosen Bhuiyan	0112330936
Md. Mahfujur Rahman Himel Akon	0112330935



Course: CSE 3422 — Software Engineering Laboratory

Trimester: Summer 2026 **Section:** I

Institution: United International University (UIU), Dhaka

Submitted to: Sayeem Been Zaman, Department of CSE

Submission Date: 28 July 2026

Contents

1	Introduction	2
2	Problem Statement	2
3	Objectives	2
4	Work Completed in Spring 2026 (Existing System)	3
5	Proposed Work for Summer 2026	4
5.1	Track A — Software engineering rigor (course deliverables)	4
5.2	Track B — Polish and publication	4
5.3	Track C — New features	5
6	Target Users and Stakeholders	6
7	Methodology	6
8	Tools and Technologies	6
9	Feasibility	7
10	Work Plan and Timeline	7
11	Team and Work Distribution	8
12	Expected Outcome	8
13	Conclusion	8

1 Introduction

University communities in Bangladesh coordinate their academic and social lives across a fragmented set of general-purpose tools: WhatsApp groups for class coordination, Facebook pages for club announcements, LinkedIn for alumni networking, and email for official notices. Information is scattered, unverified accounts mix with real ones, and nothing is aware of the distinction between a student, a faculty member, and an alumnus.

UniConnect is a private, institution-scoped social network built specifically for universities. Every account is verified through an invite-and-OTP flow against the university’s own email domain, every feature is role-aware (student, alumni, faculty, admin, and transport staff), and the platform is architected as multi-tenant SaaS so any university can be onboarded as an isolated tenant.

A functional first version of UniConnect was designed, implemented, and *deployed to production* by our team in the Spring 2026 trimester for the Web Programming course, where it grew substantially beyond the required feature set (Section 4). In Summer 2026, under the Software Engineering Laboratory, we propose to take the project through a full software-engineering lifecycle: formal requirements specification, design documentation with UML, disciplined agile iterations, systematic testing and quality assurance, and a set of new high-impact features that prepare the platform for real adoption at UIU and beyond.

2 Problem Statement

1. **Fragmentation.** Campus information lives in dozens of unofficial WhatsApp/Facebook groups with no single source of truth; important notices are routinely missed.
2. **No identity verification.** Public platforms cannot guarantee that a “UIU student” is actually a UIU student, enabling impersonation and spam.
3. **No role awareness.** Faculty announcements, alumni job referrals, and student discussions all compete in the same unstructured channels.
4. **Weak alumni linkage.** Universities lose contact with graduates, and students lose access to mentorship and referral networks.
5. **Scattered campus services.** Shuttle schedules, lost-and-found, course materials, and event registration have no common digital home.

3 Objectives

1. Apply the full software engineering process — SRS, UML-based design, agile sprint planning, code review, CI/CD, and multi-level testing — to a real, deployed product.
2. Polish the existing platform to production quality: accessibility (WCAG 2.1 AA), performance on mid-tier mobile devices, security hardening, and end-to-end test coverage.
3. Deliver a set of new features (Section 5), headlined by four substantial, one-per-member subsystems — an admin and moderation platform, live GPS shuttle tracking, an AI-generated

quiz and skill-path engine, and a lightweight academic LMS — alongside a broad grid of supporting features.

4. Publish the platform for broader use — a public landing/marketing site, demo tenant for evaluation, and documentation that allows a new university to be onboarded without developer involvement.

4 Work Completed in Spring 2026 (Existing System)

The Web Programming course required a core social-network feature set, all of which were delivered: user registration and profiles, status posts, comments, connection (friend) requests, activity notifications, a responsive newsfeed, a like/reaction system, profile picture upload, user search, and a database-driven activity log.

Beyond that requirement, the team continued development and shipped the following major subsystems, all live in production today:

Platform & security Multi-tenant architecture with strict per-university data isolation; five roles with least-privilege access; invite-only registration with email OTP; JWT access tokens with rotating refresh sessions; per-university audit logging.

Real-time layer Socket.io messaging (direct, group, and mentorship conversations), online-presence tracking with privacy tiers, and live web-push notifications — scaled across instances via Redis pub/sub.

Career & community Job board, events with registration, groups (private membership, moderator roles, shared resources, study sessions), and a LinkedIn-style profile suite (experience, education, featured work, profile analytics and viewers, vanity URLs, resume export).

Mentorship economy The most heavily iterated subsystem of the platform. A student sends a mentorship request to an alumnus, who accepts or declines it; students may also withdraw a pending request, implemented as a soft delete behind a partial unique index so the same pair can start a fresh request later. Acceptance is capacity-checked against the alumnus's `max_mentees` limit (default three, adjustable by the mentor), and immediately provisions a dedicated mentorship conversation so both parties can talk inside the platform. An accepted mentorship also relaxes the profile-privacy tier between the two users, granting the visibility the relationship implies. Two Bull background jobs drive the request lifecycle without manual intervention: a 48-hour reminder to an alumnus who has not yet responded, and a 7-day auto-expiry that transitions an unanswered request to **expired**. Once active, each meeting is logged in a dedicated sessions table — date, duration, topic and notes, editable by either party — and on completion both sides submit structured feedback. Completed sessions pay into a points economy (ten points per session, one hundred points per US dollar) that alumni redeem for gift cards, and an administrative dashboard on the mentorship page tracks mentor progress, points balances, and collected feedback.

Academic tools A lightweight LMS (course outlines, modules, assignments, gradebook), a quiz engine with answer review, and gamified learning streaks and badges.

Campus services Lost-and-found, shuttle schedules with *live GPS bus tracking* broadcast by a dedicated driver role, and automated import of official university notices from the university website (WordPress API with browser-automation fallback).

Discovery & ranking Postgres full-text search across people, posts, jobs, events, and groups; a trending “Top” feed with a Hacker-News-style hot score recomputed by background workers; hashtag explore pages; drafts and scheduled publishing.

Administration Admin dashboard with university statistics, user and role management, bulk invitations, content-report moderation, and content-sync review queues.

Infrastructure Monorepo (React + Express + shared type-safe contracts), eight background job queues, presigned direct-to-storage file uploads, and a production deployment on Azure (API), Vercel (web), and Neon (Postgres) with CI/CD from GitHub.

5 Proposed Work for Summer 2026

The Summer 2026 scope has three tracks: *engineering rigor*, *polish for publication*, and *new features*. Within the new-features track, each of the four team members owns one substantial, demoable subsystem in addition to a shared grid of smaller features.

5.1 Track A — Software engineering rigor (course deliverables)

- Software Requirements Specification (IEEE 830 style) covering the full platform.
- UML design set: use-case, class, sequence, activity, and ER diagrams; C4 architecture diagrams.
- Agile process: two-week sprints tightening to one-week sprints for the assessed update cycle, with sprint planning, backlog grooming, burndown tracking, and retrospectives (GitHub Projects).
- Quality gates: mandatory code review, CI pipeline running lint, type checks, unit and integration tests on every pull request.
- Test plan and traceability matrix; end-to-end test suite (Playwright) for the ten highest-value user journeys; target $\geq 70\%$ coverage on service-layer code.

5.2 Track B — Polish and publication

- **Accessibility audit** to WCAG 2.1 AA (contrast, keyboard navigation, screen-reader roles, reduced motion).
- **Performance hardening** for mid-tier Android devices: Lighthouse ≥ 90 , bundle-size budget, image optimization, API response caching.
- **Progressive Web App**: installable app with offline shell and cached feed, completing the existing push-notification support.
- **Security review**: rate limiting on sensitive endpoints, dependency scanning, OWASP Top-10 checklist, penetration-test style review of the multi-tenant isolation boundary.

- **Observability:** structured error tracking, uptime and performance dashboards, alerting on production incidents.
- **Localization groundwork:** Bangla language support for the highest-traffic surfaces.
- **Public launch kit:** marketing landing page, demo tenant with seeded data for evaluators, and administrator documentation.

5.3 Track C — New features

Four headline subsystems, one per owner

1. **Admin dashboard, moderation & content-sync** (Joydip Datta). University-wide statistics, bulk invitations, a content-report review queue, and automated import of official university notices via the WordPress REST API with a browser-automation fallback — imported items flow into a review queue before publication.
2. **Live GPS shuttle tracking** (Md. Mahfujur Rahman Himel Akon). A dedicated, least-privilege driver role broadcasts live location; the client interpolates the bus along its route in real time between broadcasts.
3. **AI-generated quiz engine & skill-path generation** (MD. Saem Ferdous). LLM-driven generation of quiz questions and adaptive skill paths, with model-quota fallback and timeout handling, layered on the existing academic content.
4. **Academic suite / lightweight LMS** (Md. Monabbur Hosen Bhuiyan). Course modules, assignments and submissions, and a gradebook with best- N calculation and letter grading.

Supporting features (shared across the team)

Job board; events with registration; groups with member roles, resources, and study sessions; news feed; hashtag and topic discovery; lost-and-found; online-presence tracking; web-push notifications; scheduled and draft publishing; a sticker/GIF picker for messaging; a one-click resume builder (PDF export); learning streaks and badges; and full-text search across people, posts, jobs, events, and groups.

Roadmap (stretch goal, not a committed deliverable)

AI campus assistant. A retrieval-augmented chatbot over university notices, events, shuttle schedules, and course materials (“When does the Summer registration close?”), building on the AI infrastructure already proven in production by the quiz and skill-path engine above. Pursued only if the four committed subsystems and the polish track land ahead of schedule.

6 Target Users and Stakeholders

Stakeholder	Value delivered
Students	Verified campus feed, groups and study sessions, jobs, mentorship, shuttle tracking, course tools.
Alumni	Lifelong institutional identity, mentorship rewards, referral and hiring channel.
Faculty	Role-aware announcements, course material distribution, moderated discussion.
University admin	Verified community, moderation and audit tooling, engagement analytics, notice distribution.
Transport staff	Single-purpose GPS broadcast app for live shuttle tracking.

7 Methodology

We follow **Scrum**, with the sprint cadence tied directly to the course’s assessed checkpoints (Section 10). The trimester opens with three two-week sprints running from the first class to the proposal presentation and on through the mid-term period, then switches to *one-week sprints* from 15 September so that every sprint review coincides with a graded project update. Each sprint produces a potentially shippable increment deployed to a staging environment; production releases occur after sprint review. Requirements are maintained as a prioritized product backlog derived from the SRS; every user story carries acceptance criteria that map to automated tests. GitHub flow (feature branches, pull requests, mandatory review, CI gates) governs all changes, and the definition of done includes updated documentation and passing end-to-end tests.

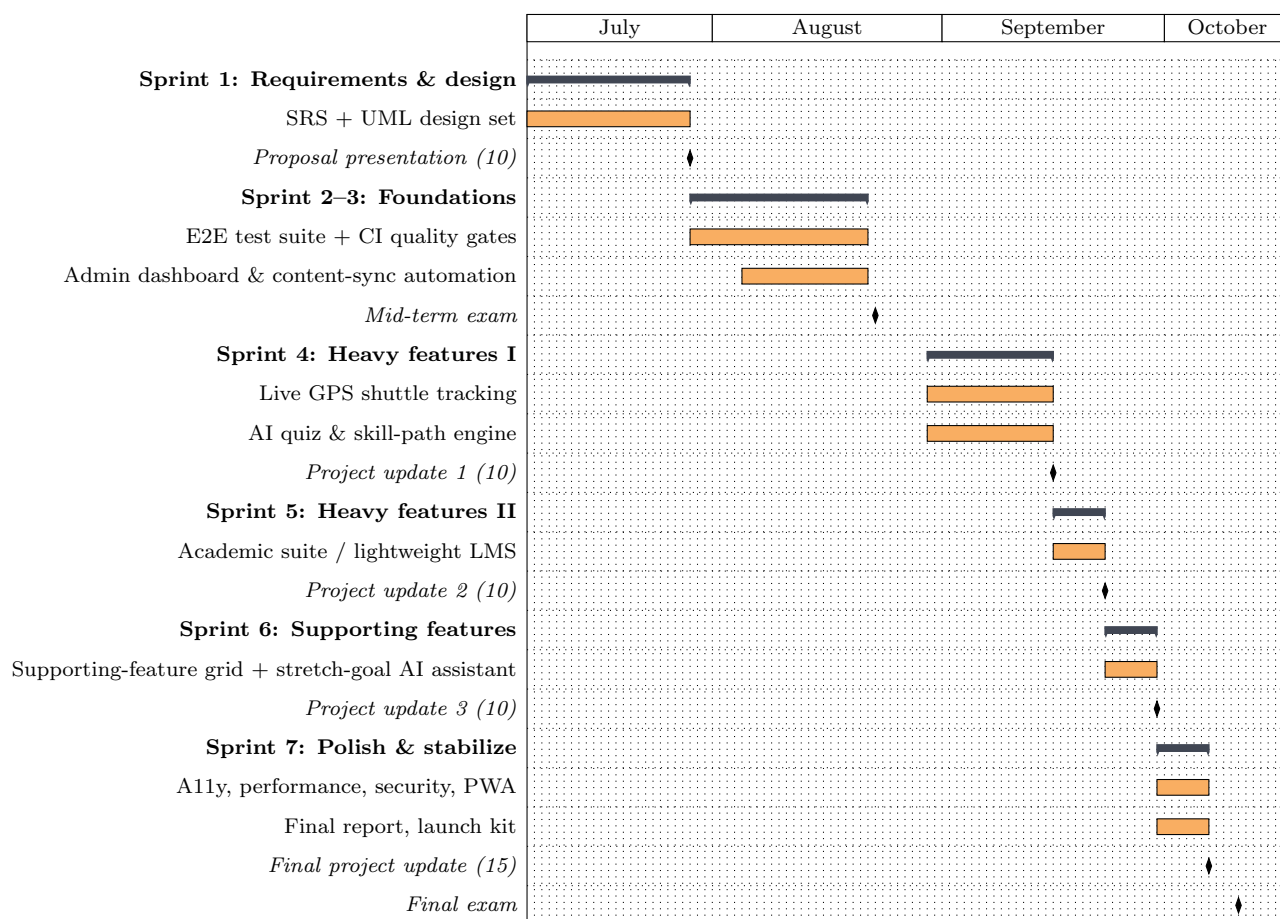
8 Tools and Technologies

Layer	Technology
Frontend	React 18, TypeScript, Vite, Tailwind CSS, TanStack Query, Zustand, Socket.io client
Backend	Node.js, Express, TypeScript, Socket.io, Zod, Knex, Bull queues
Data & infra	PostgreSQL (Neon), Redis, S3-compatible object storage, Azure App Service, Vercel
AI features	Google Generative AI (Gemini) for quiz and skill-path generation, with model-quota fallback; retrieval-augmented generation is a stretch-goal roadmap item
Engineering	GitHub Actions CI/CD, Playwright, Vitest, ESLint, Lighthouse CI, draw.io/PlantUML for UML
Process	GitHub Projects (Scrum board), LaTeX (documentation)

9 Feasibility

Technical. The riskiest foundations — multi-tenancy, real-time infrastructure, background processing, and production deployment — are already built and battle-tested in production, so Summer work concentrates on well-understood additions. The AI assistant uses managed LLM APIs rather than custom model training, keeping complexity and cost bounded. **Economic.** All infrastructure runs on free or student-tier cloud plans; the only recurring cost is metered LLM usage, which is capped per-tenant. **Operational.** The team has three trimesters of shared history on this codebase and an established CI/CD pipeline, minimizing process risk. **Schedule.** The plan is laid out against the published UIU Summer 2026 calendar and the five assessed checkpoints: it pauses during the mid-term examination week (22–29 August), sizes each sprint to end on a graded project update, and reserves the last sprint for stabilization, documentation, and presentation preparation, closing on 6 October 2026 before final examinations begin.

10 Work Plan and Timeline



The assessed deliverables and the marks attached to each are:

Deliverable	Date	Marks
Project proposal presentation	28 July 2026	10
Project update 1	15 September 2026	10
Project update 2	22 September 2026	10
Project update 3	29 September 2026	10
Final project update	6 October 2026	15
Total		55

The plan is aligned to the official UIU Summer 2026 undergraduate academic calendar — classes begin 7 July 2026, mid-term examinations run 22–29 August, the last day of course withdrawal is 9 September, and final examinations run 10–17 October. Development is deliberately paused during the mid-term week, and each sprint closes on an assessed checkpoint so that every project update presents a working, deployed increment rather than a status report.

11 Team and Work Distribution

Member	Responsibilities
Joydip Datta (leader)	Architecture, CI/CD, deployment, code review; admin dashboard, moderation, and content-sync automation
MD. Saem Ferdous	SRS and UML design documentation, requirements traceability; AI-generated quiz engine and skill-path generation
Md. Monabbur Hosen Bhuiyan	Testing lead — unit, integration and end-to-end suites; academic suite / lightweight LMS
Md. Mahfujur Rahman Himel Akon	Accessibility (WCAG 2.1 AA), PWA and offline support; live GPS shuttle tracking

12 Expected Outcome

By the end of Summer 2026 we will deliver: (1) a complete SE documentation set (SRS, UML design, test plan, traceability matrix); (2) a production platform hardened to WCAG 2.1 AA and Lighthouse ≥ 90 with automated end-to-end coverage of critical journeys; (3) four headline subsystems, one per team member — admin dashboard & moderation, live GPS shuttle tracking, an AI-generated quiz and skill-path engine, and an academic LMS — plus a broad grid of supporting features; and (4) a public launch kit enabling any university to evaluate and adopt UniConnecT. The result is not a classroom prototype but a publishable, multi-tenant SaaS product engineered with verifiable process discipline.

13 Conclusion

UniConnecT began as a course project and has already outgrown that framing: it is a deployed, multi-tenant campus platform in production use. The Software Engineering Laboratory is the natural next step — applying formal requirements, design, process, and quality engineering

to carry the product from “working” to “publishable.” We believe the combination of a real deployed system, disciplined engineering artifacts, and ambitious new features positions this project to be a standout submission for Summer 2026.

References

- [1] I. Sommerville, *Software Engineering*, 10th ed., Pearson, 2015.
- [2] IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*.
- [3] K. Schwaber and J. Sutherland, *The Scrum Guide*, 2020.
- [4] W3C, *Web Content Accessibility Guidelines (WCAG) 2.1*, 2018.
- [5] Team Mavericks, *UniConnect source repository*, GitHub, 2026.